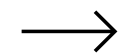
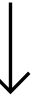




Smart Pointers

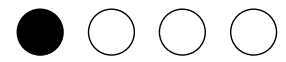
Introduction





Problématique

LES POINTEURS STANDARDS C++



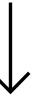
Problématique

- Les pointeurs standards C++ peuvent causer des problèmes de fuite mémoire en cas de mauvaise gestion

Exemple

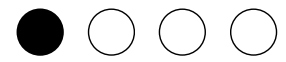
```
int doIntDivision(int x, int y){  
    int* a = new int(x);  
    if (y == 0) throw;  
    delete a;  
    return x/y;  
} // a n'est pas libéré quand une exception est levée
```





Les smart pointers

PRINCIPE



Objectif

- Les pointeurs intelligents ou smart pointers permettent de libérer automatiquement les ressources lorsqu'elles ne sont plus utilisées (out of scope).

Exemple

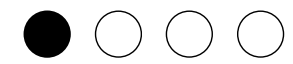
```
void doIntDivision(int x, int y){  
    std::unique_ptr<int> a = std::make_unique<int>(x);  
    if (y == 0) throw;  
    ...  
} // a sera libéré à la fin de la fonction
```





Les smart pointers

STD::UNIQUE_PTR



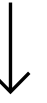
Objectif

- > Les pointeurs uniques permettent de gérer une ressource unique (pas de copie possible)

Exemple

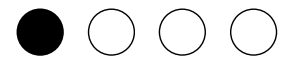
```
void doIntDivision(int x, int y){  
    // déclaration  
    std::unique_ptr<int> a;  
    // création  
    a = std::make_unique<int>(x);  
    // std::unique_ptr<int> a1 = a; /* copie impossible */  
    ...  
} // a sera libéré à la fin de la fonction
```





Les smart pointers

STD::SHARED_PTR



Objectif

- > Les pointeurs partagés permettent de partager la ressource

Exemple

```
void doIntDivision(int x, int y){  
    // déclaration  
    std::shared_ptr<int> a;  
    // création  
    a = std::make_shared<int>(x);  
    std::shared_ptr<int> a1 = a; // fonctionne  
    ...  
} // a sera libéré à la fin de la fonction
```





Les smart pointers

EXEMPLE D'UTILISATION



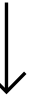
```
void foo(){
    std::unique_ptr<int> a = std::make_unique<int>(42);

    // on affiche la valeur
    std::cout << "value: " << *a << std::endl;

    // on peut récupérer le pointeur standard (pour compatibilité)
    const int* pointer = a.get();

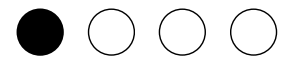
} // a sera libéré à la fin de la fonction
```





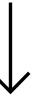
Les smart pointers

EXEMPLE D'UTILISATION



```
class Car() {  
private:  
    std::string _modelName;  
  
public:  
    Car(const std::string& name);  
    ~Car();  
    void move();  
}
```





Les smart pointers

EXEMPLE D'UTILISATION



```
int maFonctionVanilla (){
    auto car = new Car("Majorette");
    // utilisation comme un pointeur standard
    car->move();
    delete car; // libération explicite
}

int maFonctionIntelligente (){
    auto car = std::make_unique<Car>("Majorette");
    // utilisation comme un pointeur standard
    car->move();
} // La variable "car" sera libérée automatiquement
```

